

1. FUNDAMENTALS & COMMUNICATION

Programming (Coding / Software Development): The process of creating a set of instructions that tell a computer how to perform a specific task. It combines logic, problem-solving, and creativity.

Importance: Automation (eliminating repetitive tasks), Problem Solving (real-world solutions), Innovation (AI, robotics, web/mobile apps), and Foundational Skill for Data Science & Cybersecurity.

Human vs. Computer Communication

- **Human Communication:** Natural languages rich in context, emotion, and nuance.
- **Computer Communication:** Only understands binary (0s and 1s) operating on electrical signals (0 = OFF, 1 = ON).
- **Programming Languages:** Act as the bridge allowing humans to interact with computers in a structured, logical manner.

2. HIGH-LEVEL VS. LOW-LEVEL LANGUAGES

Feature	High-Level Language	Low-Level Language
Ease of Use	Easy to read & write	Difficult; requires hardware knowledge
Abstraction	High abstraction level	Low / No abstraction
Portability	Highly portable (Platform-independent)	Machine dependent (Not portable)
Performance	Lower relative speed	Extremely high performance
Control	Less hardware control	Direct hardware/register control
Dev Time	Faster development	Slower development

Low-Level Languages

- **Machine Language:** Native language of CPU consisting of binary sequences (0s and 1s). Example: 1101101010011010.
- **Assembly Language:** Uses English-like mnemonics (e.g., LOAD, ADD, STORE). Uses an **Assembler** to convert mnemonics into machine code.

Notable High-Level Languages

- **Ada:** Named after Ada Lovelace; developed for Defense Dept.
- **BASIC:** Beginner's All-purpose Symbolic Instruction Code.
- **C / C++:** C combines assembly power with portability; C++ adds Object-Oriented Programming (OOP).
- **C#:** Hybrid of Java & C++ by Microsoft.
- **COBOL & FORTRAN:** Business applications (COBOL) & scientific math (FORTRAN).
- **Java:** Platform-independent language for web, mobile, and enterprise apps.
- **Python:** General-purpose, high-level scripting language.

3. PROBLEM-SOLVING LOGIC & DESIGN

Algorithm: A finite sequence of well-defined instructions to solve a specific problem. Must be *finite*, *unambiguous*, and *correct*.

Pseudocode: Plain English representation of an algorithm mixed with programming structures; language-independent.

Flowchart: Visual diagram representing standard operational steps using specific symbols connected by flow arrows.

Standard Flowchart Symbols

Shape	Symbol Name	Function
Oval	Terminal	Start or End of process
Rectangle	Process	Calculation / Operation step
Parallelogram	Input / Output	Reading data or Printing results
Diamond	Decision	Conditional evaluation (Yes/No)
Arrow	Flowline	Direction of execution flow

4. WORKED EXAMPLES (ALGORITHM TO LOGIC)

Example 1: Sum of Two Numbers

```
Pseudocode:
BEGIN
    READ num1, num2
    COMPUTE sum = num1 + num2
    PRINT sum
END
```

Example 2: Check Even or Odd Number

```
Pseudocode:
BEGIN
    READ Number
    IF Number % 2 == 0 THEN
        PRINT "Even Number"
    ELSE
        PRINT "Odd Number"
    ENDIF
END
```

5. THE JAVA ARCHITECTURE & EXECUTION

History: Developed by James Gosling (Sun Microsystems, 1991) as *Oak*, renamed **Java** in 1995. Oracle acquired Sun in 2010.

WORA Principle: "Write Once, Run Anywhere." Source code is compiled into platform-independent bytecodes, which are executed by the platform-specific JVM.

The Java Execution Pipeline

1. Source Code (.java) → Written by Programmer
2. Compiler (javac) → Checks errors; outputs Bytecode (.class)
3. Bytecode (.class) → Platform-Independent Instruction Set
4. JVM Execution → Translates Bytecode to Native Machine Code

JDK vs. JRE vs. JVM Architecture

JDK = JRE + Development Tools (javac, debugger)

JRE = JVM + Core Class Libraries Set

Core JVM Components

- **Class Loader:** Loads, links, and initializes Java classes.
- **Execution Engine:** Includes *Interpreter & Just-In-Time (JIT) Compiler* (compiles bytecode to native machine code at runtime for high performance).
- **Garbage Collector (GC):** Automatically manages memory by deleting unreferenced objects.
- **Security Manager:** Enforces access control policies.

Java Specifications & Editions

- **JLS (Java Language Specification):** Syntax & semantics.
- **JVMS:** JVM requirements & bytecode instruction specs.
- **Java SE:** Standard Edition (Desktop/Client applications).
- **Java EE:** Enterprise Edition (Server-side: JSP, Servlets).
- **Java ME:** Micro Edition (Mobile/Embedded devices).

Popular Java IDEs

IntelliJ IDEA, Eclipse, NetBeans, BlueJ (educational), VS Code.

⚠️ COMMON STUDENT PITFALLS & EXAM TRAPS

- **JDK vs. JRE:** Running a compiled Java program needs only the JRE. Writing/compiling code requires the full JDK!
- **Flowchart Shapes:** Mixing up Parallelogram (Input/Output) with Rectangle (Process) is heavily penalized in exams!
- **Modulus Operator (%):** `x % 2 == 0` checks for Even. Remember that `x % 2` calculates the *remainder*, not the quotient.
- **File Extension Trap:** Source files are `.java`, but compiler generates binary bytecodes as `.class` files.

💡 HIGH-YIELD EXAM REVISION SHORTCUTS

- **Assembly Translator:** Assembler converts Assembly → Machine Code.
- **Java Portability Secret:** Java Bytecode is platform-independent; JVM is platform-dependent!
- **Performance Boost:** JIT Compiler in JVM optimizes hot code blocks dynamically at runtime.

6. 📄 QUICK SELF-ASSESSMENT CHECK

Q1: What is the primary role of the JIT Compiler in the JVM?

Ans: Dynamically converts frequently executed bytecode into native machine code at runtime to increase execution speed.

Q2: Differentiate between a Compiler and an Assembler.

Ans: Assembler translates assembly mnemonics into machine code; Compiler translates high-level source code into bytecode/machine code.

Q3: Write Pseudocode to display "Pass" if marks >= 50.

Ans: `READ marks; IF marks >= 50 THEN PRINT "Pass" ELSE PRINT "Fail" ENDIF`

1. ANATOMY OF A SIMPLE JAVA PROGRAM

Console Definition: An old computer term referring to the primary text entry (keyboard input) and text display device (monitor output).

```
public class Welcome {
    public static void main(String[] args) {
        // Display message Welcome to Java! on console
        System.out.println("Welcome to Java!");
    }
}
```

- **Class Declaration (Line 1):** Every Java program must have at least one class. By convention, class names start with an *Uppercase letter* (e.g., Welcome).
- **Main Method (Line 2):** public static void main(String[] args) is the official **entry point** where execution begins. A class can contain multiple methods.
- **Statements & Semicolons (Line 4):** A method contains executable statements. System.out.println(...) prints a string to console. Every statement ends with a **semicolon (;)**, called the *statement terminator*.
- **Strings:** A sequence of characters enclosed inside double quotation marks ("...").

2. COMMENTS, BLOCKS & KEYWORDS

Comments (Ignored by Compiler)

- **Line Comment:** Begins with //. Everything after // on that line is ignored.
- **Block / Paragraph Comment:** Enclosed between /* and */. All text across multiple lines inside is ignored.

Blocks & Curly Braces

A pair of curly braces { ... } forms a **block** grouping program components. Blocks can be *nested* inside each other:

- **Class Block:** Encloses data and methods belonging to the class.
- **Method Block:** Encloses statements executed by the method.

Reserved Words / Keywords

Predefined words with fixed meaning to the compiler. Cannot be used as identifiers (e.g., variable names).

Category	Java Keywords
Data Types	int, double, boolean, char, byte, short, long, float
Control Flow	if, else, switch, for, while, do
Class / Objects	class, new, this
Access Modifiers	public, private, protected
Execution / Methods	return, static, void

3. IDENTIFIERS & VARIABLES

Identifier Rules

Identifiers are user-defined names for classes, methods, variables, etc.

1. Consists of letters, digits, and underscores (_).
2. **MUST start with a letter or underscore (_).** CANNOT start with a digit (e.g., 2A is illegal).
3. Cannot be a reserved keyword or literal (true, false, null).
4. Can be of any length and is **case-sensitive** (area, Area, and AREA are 3 distinct identifiers).
5. *Best Practice:* Use descriptive names without abbreviations (e.g., numberOfStudents instead of numStuds).

Variable Declaration & Memory

Variables represent values stored in memory that can change during execution. Declaration forces the compiler to allocate memory space according to the data type.

```
datatype variableName;           // Declaration
datatype var1, var2, var3;        // Multi-declaration
int count = 1;                    // Declare + Initialize
int i = 1, j = 2;                 // Multi-initialization
```

4. JAVA DATA TYPES (PRIMITIVE VS NON-PRIMITIVE)

A data type specifies the kind of data, memory size, and valid operations.

- **Primitive Types:** Store actual value directly, fixed size, faster execution.
- **Non-Primitive Types:** Store memory references (addresses), flexible, can invoke built-in methods (e.g., String).

Type	Size	Range / Precision	Example Syntax
byte	1 B (8 bits)	−128 to 127	byte b = 100;
short	2 B (16 bits)	−32,768 to 32,767	short s = 20000;
int	4 B (32 bits)	≈ ±2 × 10 ⁹ (Default int)	int x = 50000;
long	8 B (64 bits)	Very large integers (Needs L)	long p = 80000000000L;
float	4 B (32 bits)	≈ 6–7 dec digits (Needs f)	float f = 99.99f;
double	8 B (64 bits)	≈ 15 dec digits (Default dec)	double d = 3.14159;
char	2 B (16 bits)	Single Unicode character (' ')	char c = 'A';
boolean	1 B	true or false	boolean b = true;
String	Reference	Sequence of chars (Non-primitive)	String s = "Kamal";

5. READING INPUT WITH SCANNER

Java uses System.out for standard output and System.in for standard keyboard input. Console input requires the Scanner class from java.util.

Import Types

- **Specific Import:** Imports a single explicit class.
import java.util.Scanner;
- **Wildcard Import:** Imports all classes in a package using asterisk *.
import java.util.*;

Step-by-Step Program Walkthrough (Squaring a Number)

```
import java.util.Scanner; // 1. Import Scanner

public class Test {
    public static void main(String[] args) {
        // 2. Create Scanner object bound to System.in
        Scanner input = new Scanner(System.in);

        // 3. Output prompt to user
        System.out.println("Enter the number?");

        // 4. Read integer input from console
        int number = input.nextInt();

        // 5. Compute square calculation
        int square = number * number;

        // 6. Display output result
        System.out.println("Square is : " + square);
    }
}
```

6. ⚠️ COMMON STUDENT PITFALLS & EXAM TRAPS

1. **Missing Literal Suffixes (CRITICAL TRAP):**
 - long p = 80000000000; → **Compilation Error!** Integer literal is out of range for default int. Must append L: 80000000000L.
 - float f = 5.5; → **Compilation Error!** Decimal literals default to double. Must append f: 5.5f.
2. **Reserved Keywords as Identifiers:**
 - int class = 10; → **Error!** class is a reserved keyword.
 - boolean true = false; → **Error!** Literals true, false, null cannot be identifiers.
3. **Illegal Identifiers:**
 - int 2count = 5; → Starts with digit (Illegal).
 - int total# = 10; → Contains illegal character #.
4. **Quotes Confusion:**
 - char c = "A"; → Error! Double quotes create a String, single quotes 'A' required for char.

7. 💡 HIGH-YIELD EXAM TIPS & REVISION SHORTCUTS

- **Case Sensitivity Trick:** main, Main, and MAIN are completely different. The entry point method must always be lowercase main.
- **Memory Efficiency Rule:** Use byte instead of int when handling large datasets or image raw pixels to save 75% memory (1 Byte vs 4 Bytes).
- **Statement Terminators:** Class/Method block declarations { } do NOT end with semicolons, but individual executable statements inside do!
- **Wildcard vs Specific Import Performance:** Wildcard imports (import java.util.*;) do NOT slow down runtime execution; they only slightly affect compile-time resolution.

8. 📄 SELF-ASSESSMENT QUICK CHECK

Q1: Identify all invalid identifiers from the list below:
numberOfStudents, 2ndValue, _count, class, \$price, total-amount
Answer: 2ndValue (starts with digit), class (reserved keyword), total-amount (hyphen non-allowed operator symbol).

Q2: What is the exact output and why?
int a = 10; System.out.println("Result: " + a + 5);
Answer: Result: 105 (String concatenation occurs left-to-right when combined with a String literal).

Q3: Fix the compilation errors in the following snippet:
float rate = 3.5; long dist = 90000000000;
Answer: float rate = 3.5f; long dist = 90000000000L; (Missing f and L suffixes).

Q4: How many bytes of memory are allocated for `char` vs `byte` in Java?
Answer: char = 2 Bytes (16-bit Unicode); byte = 1 Byte (8-bit signed integer range −128 to 127).

1. CORE THEORY & KEY DEFINITIONS

Assignment Statements & Expressions

An **assignment statement** designates a value for a variable. In Java, the equal sign (=) acts as the assignment operator. An assignment statement can also be evaluated as an expression.

Syntax: variable = expression;

- **Expression:** Represents a computation involving values, variables, and operators that evaluates to a single output value.
- **L-Value Rule:** The target variable must ALWAYS be placed on the left side of the = operator. Example: x = y + 1; is valid, but 1 = x; causes a compilation error.
- **Self-Referential Assignment:** Variables can appear on both sides: x = x + 1; (evaluates x + 1 first, then reassigns to x).

Constants

A **named constant** is an identifier representing a permanent value that cannot be modified after initial assignment.

Syntax: final datatype CONSTANTNAME = value;

- **Benefits:** (1) Avoids repetitive typing of identical values, (2) Allows single-location updates in source code, (3) Enhances readability.
- **Naming Convention:** Capitalize all letters with underscores separating words (e.g., PI, MAX_VALUE).

2. PRIMITIVE NUMERIC DATA TYPES

Java provides 8 primitive types; 6 are numeric (4 integer types, 2 floating-point types):

Type	Storage Size	Value Range / Description
byte	8-bit signed	-2 ⁷ to 2 ⁷ - 1 (-128 to 127)
short	16-bit signed	-2 ¹⁵ to 2 ¹⁵ - 1 (-32,768 to 32,767)
int	32-bit signed	-2 ³¹ to 2 ³¹ - 1 (≈ -2.14 × 10 ⁹ to 2.14 × 10 ⁹)
long	64-bit signed	-2 ⁶³ to 2 ⁶³ - 1 (≈ -9.22 × 10 ¹⁸ to 9.22 × 10 ¹⁸)
float	32-bit IEEE 754	Single precision (≈ 6-7 decimal digits precision)
double	64-bit IEEE 754	Double precision (≈ 15 decimal digits; preferred type)

Reading Numbers with Scanner

Located in java.util.Scanner. Key input methods:

- nextByte(), nextShort(), nextInt(), nextLong()
- nextFloat(), nextDouble()

3. ARITHMETIC OPERATORS & MODULO RULES

Op	Meaning	Example	Result
+	Addition	34 + 1	35
-	Subtraction / Negation	34.0 - 0.1	33.9
*	Multiplication	300 * 30	9000
/	Division	1.0 / 2.0	0.5
%	Remainder (Modulo)	20 % 3	2

Division Rules (Integer vs. Floating-Point)

- **Integer Division:** When *both* operands are integers, fractional parts are truncated (chopped off, NOT rounded).
Examples: 5 / 2 → 2; -5 / 2 → -2
- **Floating-Point Division:** If *at least one* operand is floating-point, standard decimal division occurs.
Example: 5.0 / 2 → 2.5

Modulo Operator (%) Special Rules

Computes the remainder of dividend % divisor.

- **Sign Rule:** The remainder carries the sign of the **DIVIDEND ONLY** (the left operand).
7 % 3 = 1 | 3 % 7 = 3 | 12 % 4 = 0 | 26 % 8 = 2
- -7 % 3 = -1 (Dividend negative → Result negative)
- -26 % -8 = -2 (Dividend negative → Result negative)
- 20 % -13 = 7 (Dividend positive → Result positive)

Unary vs. Binary Operators

- **Unary:** Takes 1 operand. Example: -5 (negation operator).
- **Binary:** Takes 2 operands. Example: 4 - 5 (subtraction operator).

4. STEP-BY-STEP WORKED EXAMPLES

Example 1: Compute Area of Circle (\$A = \pi r^2\$)

```
import java.util.Scanner;

public class ComputeAreaWithConstant {
    public static void main(String[] args) {
        final double PI = 3.14159; // Named Constant
        Scanner input = new Scanner(System.in);

        System.out.print("Enter a number for radius: ");
        double radius = input.nextDouble();

        double area = radius * radius * PI; // Computation

        System.out.println("The area for the circle of radius "
            + radius + " is " + area);
    }
}
```

Example 2: Time Conversion (Seconds → Mins + Secs)

Problem: Convert total input seconds (e.g., 500s) into minutes and remaining seconds.

```
import java.util.Scanner;

public class DisplayTime {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Enter an integer for seconds: ");
        int seconds = input.nextInt();

        int minutes = seconds / 60; // Integer Division
        int remainingSeconds = seconds % 60; // Modulo

        System.out.println(seconds + " seconds is " + minutes
            + " minutes and " + remainingSeconds + " seconds");
    }
}
```

Trace for 500s: 500 / 60 = 8 mins, 500 % 60 = 20 remaining secs → **8 mins 20 secs.**

⚠️ COMMON STUDENT PITFALLS & EXAM TRAPS

- **L-Value Misassignment Trap:** Writing 10 = x; or x + y = z; is a compiler error. Left side must be a variable.
- **Integer Division Truncation Trap:** Writing double ratio = 1 / 2; evaluates 1 / 2 as integer division first (0), then assigns 0.0 to ratio! Use 1.0 / 2.0 or (double)1 / 2 to get 0.5.
- **Modulo Sign Misunderstanding:** Remembering that 20 % -13 yields 7 (NOT -7) because sign follows the dividend (20).
- **Constant Re-assignment:** Trying to update a final variable like PI = 3.14; will break compilation.

💡 HIGH-YIELD EXAM TIPS & REVISION SHORTCUTS

- **Small Dividend Rule:** If a < b (where a, b > 0), then a % b = a (e.g., 3 % 7 = 3).
- **Float vs Double Default:** Floating-point literals (e.g., 3.14) are treated as double by default in Java.
- **Data Type Selection:** Always prefer int for whole numbers and double for decimals unless memory-restricted.

5. SELF-ASSESSMENT QUICK CHECK

1. **Q: What is the output of System.out.println(-17 % 5);?**
Ans: -2 (17 divided by 5 gives remainder 2; sign matches dividend -17).
2. **Q: What is the value stored in result: double result = 5 / 2;?**
Ans: 2.0 (Integer division 5 / 2 gives 2, converted to double).
3. **Q: Identify the errors in: final int MAX = 10; MAX = 20; 100 = MAX;**
Ans: (1) Cannot reassign value to final variable MAX. (2) 100 = MAX violates L-Value syntax rule.

1. CORE THEORY & DEFINITIONS

A. Augmented Assignment Operators

In programming, updating a variable based on its existing value (e.g., `count = count + 1;`) is extremely common. Java provides **augmented (compound) assignment operators** that combine arithmetic operations with assignment.

Operator	Name	Example	Equivalent Expression
<code>+=</code>	Addition assignment	<code>i += 8</code>	<code>i = i + 8</code>
<code>-=</code>	Subtraction assignment	<code>i -= 8</code>	<code>i = i - 8</code>
<code>*=</code>	Multiplication assignment	<code>i *= 8</code>	<code>i = i * 8</code>
<code>/=</code>	Division assignment	<code>i /= 8</code>	<code>i = i / 8</code>
<code>%=</code>	Remainder assignment	<code>i %= 8</code>	<code>i = i % 8</code>

B. Increment and Decrement Operators

Special unary operators used to increase or decrease a variable's value by exactly 1.

Operator	Type	Example (for <i>i</i> = 1)	Description & Action
<code>++var</code>	Preincrement	<code>int j = ++i;</code>	Increments <i>i</i> by 1 first, then returns new value (<i>j</i> = 2, <i>i</i> = 2).
<code>var++</code>	Postincrement	<code>int j = i++;</code>	Returns original <i>i</i> value first, then increments <i>i</i> by 1 (<i>j</i> = 1, <i>i</i> = 2).
<code>--var</code>	Predecrement	<code>int j = --i;</code>	Decrements <i>i</i> by 1 first, then returns new value (<i>j</i> = 0, <i>i</i> = 0).
<code>var--</code>	Postdecrement	<code>int j = i--;</code>	Returns original <i>i</i> value first, then decrements <i>i</i> by 1 (<i>j</i> = 1, <i>i</i> = 0).

C. Numeric Type Conversions & Casting

Type Casting: Operations converting a value from one data type to another.

- Widening (Implicit):** Converting a smaller range type to a larger range (e.g., `int` to `double`). Done *automatically* by Java. If an `int` and `double` interact, the `int` widens to `double` (e.g., $3 \times 4.5 \rightarrow 3.0 \times 4.5 = 13.5$).
- Narrowing (Explicit):** Converting a larger range type to a smaller range (e.g., `double` to `int`). Requires explicit casting syntax (`targetType`)value because potential data loss can occur.

```
System.out.println((int)1.7); // Output: 1 (Fractional part TRUNCATED)
System.out.println((double)1/2); // Output: 0.5 (1 cast to 1.0, then 1.0 / 2)
System.out.println(1/2); // Output: 0 (Integer division truncates result)
```

D. Relational & Logical Operators

Relational Operators: Compare two values and return a boolean (true or false).

Java	Math	Name	Example (<i>radius</i> = 5)	Result
<code><</code>	<code><</code>	Less than	<code>radius < 0</code>	false
<code><=</code>	<code>≤</code>	Less or equal	<code>radius <= 0</code>	false
<code>></code>	<code>></code>	Greater than	<code>radius > 0</code>	true
<code>>=</code>	<code>≥</code>	Greater or equal	<code>radius >= 0</code>	true
<code>==</code>	<code>=</code>	Equal to	<code>radius == 0</code>	false
<code>!=</code>	<code>≠</code>	Not equal to	<code>radius != 0</code>	true

Logical Operators: Combine multiple Boolean expressions.

- `!` (NOT) - Negates truth value.
- `&&` (AND) - True ONLY if BOTH operands are true.
- `||` (OR) - True if AT LEAST ONE operand is true.
- `^` (XOR / Exclusive OR) - True if operands have DIFFERENT truth values.

2. COMPLETE TRUTH TABLES

<i>p</i> ₁	<i>p</i> ₂	! <i>p</i> ₁	<i>p</i> ₁ && <i>p</i> ₂	<i>p</i> ₁ <i>p</i> ₂	<i>p</i> ₁ ^ <i>p</i> ₂
false	false	true	false	false	false
false	true	true	false	true	true
true	false	false	false	true	true
true	true	false	true	true	false

3. STEP-BY-STEP WORKED EXAMPLES

Example 1: Pre vs Post Increment Evaluation

```
int i = 10;
int newNum = 10 * i++;
// Execution: 10 * 10 = 100 assigned to newNum, then i becomes 11.
// Output: i is 11, newNum is 100

int i = 10;
int newNum = 10 * (++i);
// Execution: i increments to 11 first, then 10 * 11 = 110 assigned to newNum.
// Output: i is 11, newNum is 110
```

Example 2: Sales Tax Currency Rounding

```
import java.util.Scanner;
public class SalesTax {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter purchase amount: "); // User enters: 197.55
        double purchaseAmount = input.nextDouble();
        double tax = purchaseAmount * 0.06; // 197.55 * 0.06 = 11.853
        System.out.println("Sales tax is $" + (int)(tax * 100) / 100.0);
    }
}
```

Step-by-step Trace:

- `tax * 100 = 11.853 × 100 = 1185.3`
- `(int)(1185.3) = 1185` (fraction truncated)
- `1185 / 100.0 = 11.85` (double division retains 2 decimal places)
- Final Output:** Sales tax is \$11.85

Example 3: Logical Operators Trace (Inputs 6 & 15)

```
int number = input.nextInt();
if (number % 2 == 0 && number % 3 == 0)
    System.out.println(number + " is divisible by 2 and 3.");
if (number % 2 == 0 || number % 3 == 0)
    System.out.println(number + " is divisible by 2 or 3.");
if (number % 2 == 0 ^ number % 3 == 0)
    System.out.println(number + " is divisible by 2 or 3, but not both.");
```

Evaluation for Input = 6:

- `6 % 2 == 0` (true), `6 % 3 == 0` (true)
- `true && true` → **Prints:** 6 is divisible by 2 and 3.
- `true || true` → **Prints:** 6 is divisible by 2 or 3.
- `true ^ true` → false (No output).

Evaluation for Input = 15:

- `15 % 2 == 0` (false), `15 % 3 == 0` (true)
- `false && true` → false (No output).
- `false || true` → **Prints:** 15 is divisible by 2 or 3.
- `false ^ true` → **Prints:** 15 is divisible by 2 or 3, but not both.

Example 4: Leap Year Algorithm Implementation

```
import java.util.Scanner;
public class LeapYear {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter a year: ");
        int year = input.nextInt();
        // A year is leap if (divisible by 4 AND NOT 100) OR (divisible by 400)
        boolean isLeapYear = (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
        System.out.println(year + " is a leap year? " + isLeapYear);
    }
}
```

⚠ Common Student Pitfalls & Exam Traps:

- Casting vs Rounding:** `(int)1.99` evaluates to 1, NOT 2. Direct type casting truncates decimals without rounding!
- Assignment vs Equality:** Writing `if (a = b)` is a syntax error in Java (unlike C++). Always use `if (a == b)` for comparison.
- Integer Division Pitfall:** `1 / 2` evaluates to 0 because both operands are integers. To get 0.5, at least one operand must be double: `1.0 / 2` or `(double)1 / 2`.
- XOR Operator Trap:** Remember $A \wedge B$ is false when both *A* and *B* are true. It is ONLY true when exactly one operand is true.

💡 High-Yield Exam Tips & Revision Shortcuts:

- Operator Precedence Order:**
 - Unary Operators (`++`, `--`, `!`, `(type)`)
 - Multiplicative (`*`, `/`, `%`)
 - Additive (`+`, `-`)
 - Relational (`<`, `<=`, `>`, `>=`)
 - Equality (`==`, `!=`)
 - Logical (`^` → `&&` → `||`)
 - Compound Assignments (`+=`, `-=`, etc.)
- Truncating Trick:** To keep *N* decimal places using integer casting: $\text{Result} = \frac{\lfloor x \times 10^N \rfloor}{10^N}$ (as double).

4. 📝 SELF-ASSESSMENT QUICK CHECK

- What is the value of *a* and *b* after: `int a=5; int b = ++a + ++a;`?
Answer: *a*=7, *b*=12 (`++a` uses 5, increments *a* to 6; `++a` increments *a* to 7, uses `7.5 + 7 = 12`).
- Evaluate: `(double)(5 / 2)` vs `(double)5 / 2`.
Answer: `(double)(5/2) → 2.0`; `(double)5/2 → 2.5`.
- State the condition for a Leap Year in Java syntax.
Answer: `(year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)`

1. OVERVIEW OF SELECTION STATEMENTS

Definition: Control structures specify the order in which statements are executed. Selection statements alter sequential execution by choosing alternative paths based on a boolean condition (true or false).

6 Types of Selection Statements in Java

- 1. One-way if statement: Executes an action if condition is true.
- 2. Two-way if-else statement: Chooses between two actions.
- 3. Nested if statement: An if placed inside another if.
- 4. Multi-way if-else statement: Evaluates multiple sequential conditions.
- 5. switch statement: Selects execution branch based on an expression value.
- 6. Conditional expression: Ternary shorthand operator (? :).

2. SINGLE & DUAL BRANCH SELECTION

One-Way if Statements

Executes statement(s) if and only if the boolean expression evaluates to true. If false, control passes directly to the next statement.

```
if (boolean-expression) {
    statement(s);
}
```

Two-Way if-else Statements

Executes true-case statements if boolean expression is true; otherwise, false-case statements are executed.

```
if (boolean-expression) {
    statement(s)-for-true-case;
} else {
    statement(s)-for-false-case;
}
```

3. MULTI-WAY SELECTION & NESTING

Selection statements can be nested with no theoretical depth limit. Preferred form for sequential ranges is a multi-way if-else chain.

```
// Academic Score Grading Example
if (score >= 90.0) {
    System.out.print("A");
} else if (score >= 80.0) {
    System.out.print("B");
} else if (score >= 70.0) {
    System.out.print("C");
} else if (score >= 60.0) {
    System.out.print("D");
} else {
    System.out.print("F");
}
```

4. THE SWITCH STATEMENT

A switch statement simplifies multi-way branching based on a discrete value comparison rather than ranges.

```
switch (switch-expression) {
    case value1:
        statement(s)1;
        break; // Optional: prevents fall-through
    case value2:
        statement(s)2;
        break;
    case valueN:
        statement(s)N;
        break;
    default: // Optional: executed if no match
        statement(s)-for-default;
}
```

Mandatory Rules for switch Statements

- Allowed Types: switch-expression MUST evaluate to char, byte, short, int, or String (and enums). Floating-point types (double, float) and boolean are STRICTLY ILLEGAL.
- Constant Cases: value1 ... valueN must be constant expressions (literals or final variables). They cannot contain non-constant variables (e.g., 1 + x is illegal).
- Type Matching: Case label values must match the data type of the switch-expression.
- Fall-Through Behavior: When a match occurs, statements execute sequentially until a break or end of switch is reached. Omitting break causes execution to "fall through" to subsequent cases!
- Parentheses: The switch-expression must always be enclosed in parentheses.

5. STEP-BY-STEP WORKED EXAMPLES

Example 1: Body Mass Index (BMI) Calculation

Formula for Body Mass Index:

$BMI = \frac{\text{weight (kg)}}{\text{height (m)}^2}$	
BMI Value	Health Interpretation
BMI < 18.5	Underweight
18.5 ≤ BMI < 25.0	Normal
25.0 ≤ BMI < 30.0	Overweight
30.0 ≤ BMI	Obese

```
double bmi = weight / (height * height);
if (bmi < 18.5)
    System.out.println("Underweight");
else if (bmi < 25.0)
    System.out.println("Normal");
else if (bmi < 30.0)
    System.out.println("Overweight");
else
    System.out.println("Obese");
```

Example 2: Rewriting if-else to switch

Problem: Find x when a = 3, then rewrite using switch:

```
int x = 1, a = 3;
if (a == 1)    x += 5;
else if (a == 2) x += 10;
else if (a == 3) x += 15;
else if (a == 4) x += 20;
```

Trace: Since a = 3, condition a == 3 is true → x = 1 + 15 = 16.

```
// Equivalent switch implementation:
int x = 1, a = 3;
switch (a) {
    case 1: x += 5; break;
    case 2: x += 10; break;
    case 3: x += 15; break;
    case 4: x += 20; break;
}
```

6. COMMON STUDENT PITFALLS & EXAM TRAPS

- 1. Premature Semicolon Trap (CRITICAL EXAM TRAP):
if (radius >= 0); // Premature semicolon ends 'if'!
{
 area = radius * radius * PI; // Executes unconditionally!
}

2. Missing Block Braces {} on Multi-line Blocks:
if (radius >= 0)
 area = radius * radius * PI;
 System.out.println("Area: " + area); // Always executes!

Only the first statement belongs to if without {}.

3. Dangling else Ambiguity: An else clause always pairs with the most recent unmatched if in the same block, regardless of indentation.

4. Illegal Switch Types & Case Variables:
• switch (doubleValue) → Compilation Error! double / float / boolean not allowed.
• case x + 1: → Compilation Error! Case labels must be compile-time constants, not variables.

7. HIGH-YIELD EXAM TIPS & REVISION SHORTCUTS

- Boolean Redundancy Shortcut:
Avoid if (even == true). Write if (even) directly.
Instead of if (n % 2 == 0) even = true; else even = false;; use:
boolean even = (number % 2 == 0);

• Factoring Duplicate Output Code: Remove common statements repeated inside both if and else blocks and place them after the conditional structure to clean up code.

• Nested Range Order: When writing multi-way if-else for numeric ranges, conditions must be ordered strictly sequentially (e.g., highest to lowest or lowest to highest) to avoid unreachable dead code branches.

8. SELF-ASSESSMENT QUICK CHECK

Q1: What is the output of the following code snippet?

```
int x = 2; switch(x) { case 1: x+=1; case 2: x+=2; case 3: x+=3; default: x+=4; }
System.out.println(x);
```

Answer: 11 (Fall-through occurs! Matches case 2: x = 2 + 2 = 4; falls through to case 3: x = 4 + 3 = 7; falls through to default: x = 7 + 4 = 11).

Q2: Why does `switch (score >= 60)` cause a syntax error?

Answer: `score >= 60` yields a `boolean` value (`true`/`false`), which is an illegal type for a `switch` expression in Java.

Q3: To which `if` statement does the `else` belong?

```
if (i > j) if (i > k) System.out.println("A"); else System.out.println("B");
```

Answer: It belongs to `if (i > k)` because Java pairs an `else` with the nearest unmatched `if` clause.

Q4: Calculate BMI for weight = 70 kg and height = 1.75 m, and state the category.

Answer: BMI = 70 / (1.75 × 1.75) = 70 / 3.0625 ≈ 22.86 kg/m². Category: **Normal** (18.5 ≤ 22.86 < 25.0).

1. INTRODUCTION TO REPETITION STATEMENTS

A **loop** (repetition statement) controls program execution by repeatedly executing a block of code while a specified boolean condition evaluates to true.

- **Loop Body:** The block of code executed repeatedly inside the loop structure.
- **Iteration:** A single, complete execution pass through the loop body.
- **Loop-Continuation-Condition:** A boolean expression evaluated before or after each iteration that determines whether the loop continues or terminates.

Categories of Loop Control Mechanisms

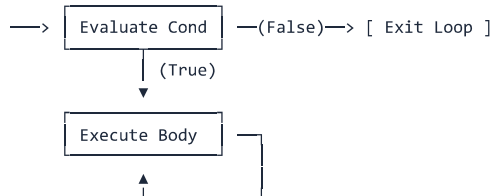
1. **Counter-Controlled Loop:** Used when the exact number of iterations is known prior to loop execution. Employs a control variable (counter).
2. **Sentinel-Controlled Loop:** Used when the number of iterations is indefinite. Execution continues until a special input signal (a *sentinel value*) is encountered.

2. THE WHILE LOOP STRUCTURE (PRETEST)

The while loop is a **pretest loop**: it tests the continuation condition *before* executing the loop body. If the condition is initially *false*, the body is never executed (0 iterations).

syntax:
while (loop-continuation-condition) {
 // Loop body (Statements)
}

[Flowchart Logic]



Counter-Controlled while Example

```
int count = 0; // Initialize control variable
while (count < 100) { // Continuation condition
    System.out.println("Welcome to Java!");
    count++; // Update control variable
}
```

Sentinel-Controlled Case Study: GuessNumber

Generates a random integer $n \in [0, 100]$ and prompts the user until guessed correctly.

```
// Correct Sentinel Implementation
int number = (int)(Math.random() * 101);
Scanner input = new Scanner(System.in);
int guess = -1; // Initialized to non-matching value

while (guess != number) { // Sentinel check
    System.out.print("Enter your guess: ");
    guess = input.nextInt();
    if (guess == number)
        System.out.println("Yes, the number is " + number);
    else if (guess > number)
        System.out.println("Your guess is too high");
    else
        System.out.println("Your guess is too low");
}
```

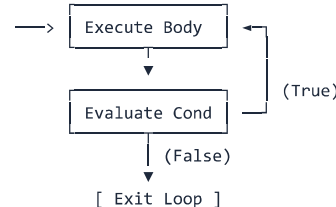
Logic Note: Initializing `guess = -1` guarantees entry into the loop because `number ≥ 0`.

3. THE DO-WHILE LOOP STRUCTURE (POSTTEST)

The do-while loop is a **posttest loop**. It executes the loop body first, then tests the continuation condition. It is guaranteed to execute **at least once** (≥ 1 iteration).

syntax:
do {
 // Loop body (Statements)
} while (loop-continuation-condition); // Note trailing semicolon!

[Flowchart Logic]



Sentinel Accumulator with do-while

```
Scanner input = new Scanner(System.in);
int data, sum = 0;
do {
    System.out.print("Enter int (0 ends): ");
    data = input.nextInt();
    sum += data;
} while (data != 0); // Terminate when sentinel (0) entered
System.out.println("The sum is " + sum);
```

4. THE FOR LOOP STRUCTURE (PRETEST)

The for loop provides a compact syntax designed for counter-controlled iteration, combining initialization, condition checking, and updates in a single statement header.

syntax:
for (initial-action; loop-condition; action-after-iteration)
{
 // Loop body
}

Execution Order Sequence

1. Execute initial-action (runs exactly once at loop start).
2. Evaluate loop-condition. If false, terminate loop.
3. If true, execute the loop body.
4. Execute action-after-iteration (e.g., counter update).
5. Repeat steps 2 through 4.

```
// Standard Counter Iteration
for (int i = 0; i < 100; i++) {
    System.out.println("Welcome to Java!");
}
```

Advanced for Loop Features

- **Control Variable Scope:** If declared inside the header (e.g., `int i = 0`), `i` exists only within the loop. If declared prior to the loop, it remains accessible after loop termination.
- **Comma Expressions:** Multiple initializations or updates can be combined using commas:
`for (int i = 0, j = 10; i + j < 10; i++, j--) { ... }`
- **Omitted Expressions:** All three parts are optional. Omitting the condition defaults to true.

5. LOOP EQUIVALENCE & CONVERSIONS

All three loop constructs (while, do-while, for) are expressively equivalent. Any while loop can be converted into a for loop and vice-versa.

Standard Transformation Rules

while Construct	Equivalent for Construct
<pre>initial-action; while (condition) { // Body action-after; }</pre>	<pre>for (initial-action; condition; action-after) { // Body }</pre>
<pre>while (condition) { // Body }</pre>	<pre>for (; condition;) { // Body }</pre>

Infinite Loop Variants

<pre>// (a) Omitted condition Preferred form for (;;) { true; } { // Infinite body Infinite body }</pre>	<pre>(b) Explicit true while (true) { // Infinite body }</pre>	<pre>(c) for (; true;) { // Infinite body }</pre>
--	--	--

6. NESTED LOOPS & MULTIDIMENSIONAL TABLES

A loop placed inside another loop is a **nested loop**. For each single pass of the outer loop, the inner loop completes all of its iterations.

```
// 9x9 Multiplication Table Algorithm
for (int i = 1; i <= 9; i++) { // Outer loop (rows)
    for (int j = 1; j <= 9; j++) { // Inner loop (columns)
        System.out.printf("%4d", i * j);
    }
    System.out.println(); // Newline after row completes
}
```

7. HIGH-YIELD CODE EXECUTION TRACES

Trace 1: Maximum Finder (Input Sequence: 2 3 4 5 0)

```
int number = input.nextInt(); // Reads 2
int max = number; // max = 2
while (number != 0) {
    number = input.nextInt(); // Reads 3, 4, 5, 0
    if (number > max) max = number;
}
System.out.println("max is " + max); // Output: max is 5
System.out.println("number " + number); // Output: number 0
```

Trace 2: Pre-increment vs Post-increment in Header

Comparing for (int i=0; i<10; ++i) vs for (int i=0; i<10; i++):

- Both loops execute the body **10 times** ($i = 0, 1, \dots, 9$).
- Both produce the exact same final sum because the update step occurs *after* the body completes.

Trace 3: Accumulator Loop (Input Sequence: 2 3 4 5 0)

```
int number, sum = 0, count;
for (count = 0; count < 5; count++) {
    number = input.nextInt();
    sum += number;
}
// Reads 5 tokens: 2 + 3 + 4 + 5 + 0 = 14
// Output: sum is 14, count is 5
```

Common Student Pitfalls & Exam Traps:

- The Trailing Semicolon Bug:**
Placing a `;` right after a loop header creates an empty loop body:
while (i < 10); { i++; } → Infinite loop because i is never updated inside the empty loop!
- Off-by-One Error (OBOE):**
Confusing `<` with `<=`.
for(int i=0; i<=10; i++) runs ****11 times****, not 10.
- Floating-Point Loop Counters:**
Avoid using floating-point variables as counters (e.g., `for(double x=0; x!=1.0; x+=0.1)`) due to precision errors making `x == 1.0` evaluate to `false`.
- Scope Leak Errors:**
Attempting to access i outside `for(int i=0;...)` causes a compiler error.
- Missing `do-while` Semicolon:**
Forgetting the required ending `;` after `while(cond)` in a `do-while` statement.

High-Yield Exam Tips & Revision Shortcuts:

- Loop Selection Tree:**
 - Count known advance? → Use for loop.
 - Indefinite count, check first? → Use while loop.
 - Must run at least once? → Use do-while loop.
- Iteration Count Formula:**
For `for (int i = a; i < b; i += c)`:

$$\text{Iterations} = \left\lceil \frac{b-a}{c} \right\rceil \quad (\text{if } b > a)$$

8. SELF-ASSESSMENT QUICK CHECK

1. Q: How many times does the following loop print "Hello"?

```
int c = 10;
while (c > 0) { System.out.println("Hello"); c -= 3; }
```

Solution: Trace $c = 10, 7, 4, 1 \rightarrow$ Prints **4 times**. Next $c = -2 \leq 0$ (terminates).

2. Q: Convert this while loop to an equivalent for loop:

```
int x = 5; while (x <= 50) { sum += x; x += 5; }
```

Solution:

```
for (int x = 5; x <= 50; x += 5) { sum += x; }
```

3. Q: What is the output of the following nested loop code?

```
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= i; j++) System.out.print(j);
}
```

Solution:

Outer $i = 1$: Inner $j = 1 \rightarrow$ `1`
 Outer $i = 2$: Inner $j = 1, 2 \rightarrow$ `12`
 Outer $i = 3$: Inner $j = 1, 2, 3 \rightarrow$ `123`
 Final Output: `112123`

4. Q: What is the value of count after the loop ends?

```
int count = 0;
for (int i = 0; i < 5; i++)
    for (int j = 0; j < 3; j++) count++;
```

Solution: Outer iterations $\times$ Inner iterations $= 5 \times 3 = 15$. count = 15.

1. CORE THEORY & KEY DEFINITIONS

A **method** (or function) is a collection of grouped programming statements designed to perform a specific operation, simplify complex problems, eliminate code redundancy, and promote code reusability.

Why Use Methods?

Instead of repeating similar logic across multiple locations (e.g., calculating range sums $1 \rightarrow 10$, $20 \rightarrow 37$, $35 \rightarrow 49$), methods allow programmers to write the common algorithm once and execute it dynamically with different inputs.

Anatomy of a Java Method Definition

A standard method definition comprises six core elements:

<pre>modifier returnType methodName(parameterList) { // Method body (Statements) return value; // Required if returnType != void }</pre>	
Element	Description & Architectural Role
Modifier	Defines access control and scope behavior (e.g., public static).
Return Type	Data type returned (int, double, etc.) or void if no value is returned.
Method Name	Identifier used to call the method (e.g., max, calculateFare).
Parameters	Formal Parameters: Variables declared in the method header serving as placeholders.
Signature	Combines methodName + parameterList (data types, order, quantity).
Method Body	Encapsulated executable logic enclosed in braces { }.

2. KEY SYNTAX & TECHNICAL RULES

Formal Parameters vs. Actual Parameters (Arguments)

- **Formal Parameters:** Local variables declared in the method header (e.g., int num1, int num2).
- **Actual Parameters (Arguments):** Concrete values or expressions passed during invocation (e.g., max(5, 10) or max(x, y)).

Method Classification

1. **Value-Returning Methods:** Returns a result using the return keyword. A call to this method is treated as a value in expressions:
int result = max(3, 4);
2. **Void Methods:** Performs an action without returning a value (return type void). A call must be a standalone statement:
System.out.println("Hello");

Flow of Execution & Control Transfer

When a program calls a method, control transfers to the invoked method. Execution resumes in the caller method when a return statement is encountered or the closing brace } is reached.

3. STEP-BY-STEP WORKED EXAMPLES

Example 1: Sum of Integers in Range $[n_1, n_2]$

Formula: $\text{Sum} = \sum_{i=\text{num1}}^{\text{num2}} i$

```
public static int sum(int num1, int num2) {  
    int result = 0;  
    for (int i = num1; i <= num2; i++) {  
        result += i;  
    }  
    return result; // Return accumulated total  
}  
// Invocation in main:  
System.out.println("Sum 1..10: " + sum(1, 10)); // 55  
System.out.println("Sum 20..37: " + sum(20, 37)); // 513
```

Example 2: Temperature Unit Conversion

Conversion Formula: $F = (C \times 1.8) + 32$

```
public static double convertToFahrenheit(double celsius) {  
    return (celsius * 1.8) + 32.0;  
}  
// Step-by-Step Numerical Substitution for C = 25.5°C:  
// F = (25.5 * 1.8) + 32.0 = 45.9 + 32.0 = 77.9°F
```

Example 3: Maximum Value Selection Logic

```
public static int max(int num1, int num2) {  
    if (num1 > num2)  
        return num1;  
    else  
        return num2;  
}
```

4. LECTURE EXERCISES & SOLUTIONS

Exercise 1: Grade Evaluation (Void Method)

```
public static void printGrade(double score) {  
    if (score >= 90.0) System.out.println('A');  
    else if (score >= 80.0) System.out.println('B');  
    else if (score >= 70.0) System.out.println('C');  
    else if (score >= 60.0) System.out.println('D');  
    else System.out.println('F');  
}  
// Test cases: printGrade(68.6) -> D | 92.3 -> A | 53.9 -> F
```

Exercise 2: Scope Analysis & Bug Fixing

Scope Error Analysis: In the slide example, variable finalSalary was defined inside calculateTotalSalary(). Attempting to access it inside main() causes a compiler error: cannot find symbol - variable finalSalary.

// Corrected Code: Return value or declare in main
public static double calculateTotalSalary(int basicSalary) {
 return basicSalary + 200.0; // Local variable scope isolated
}

Exercise 3: Argument Order Mismatch

```
public static void printStudentInfo(String name, int age) {}  
// INCORRECT: printStudentInfo(20, "Alice");  
// Reason: Arguments MUST strictly match parameters in type,  
// order, and number. (int cannot implicitly cast to String)
```

Exercise 4: Multi-Method Application (CityCabSystem)

```
import java.util.Scanner;  
  
public class CityCabSystem {  
    public static void displayMainMenu() {  
        System.out.println("1. Calculate New Trip Fare\n2. Exit");  
    }  
  
    public static int getRatePerVehicle(int choice) {  
        switch (choice) {  
            case 1: return 60; // TukTuk  
            case 2: return 90; // Sedan  
            case 3: return 150; // VIP Van  
            default: return -1; // Invalid  
        }  
    }  
  
    public static double calculateFare(String name, double dist, int rate) {  
        double gross = dist * rate;  
        if (gross > 1000.0) gross -= 100.0; // Rs. 100 Discount  
        return gross;  
    }  
  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        while (true) {  
            displayMainMenu();  
            int opt = sc.nextInt();  
            if (opt == 2) break;  
            if (opt == 1) {  
                System.out.print("Name: "); String name = sc.next();  
                System.out.print("Distance (km): "); double dist = sc.nextDouble();  
                System.out.print("Vehicle (1-Tuk, 2-Sedan, 3-Van): ");  
                int vChoice = sc.nextInt();  
                int rate = getRatePerVehicle(vChoice);  
                if (rate == -1) {  
                    System.out.println("Invalid Vehicle Choice!");  
                    continue;  
                }  
                double finalFare = calculateFare(name, dist, rate);  
                System.out.println("Receipt for " + name + ": Rs. " + finalFare);  
            }  
        }  
    }  
}
```

5. COMMON STUDENT TRAPS & PITFALLS

1. **Parameter Syntax Error:** Writing max(int num1, num2) is illegal. Every parameter MUST explicitly declare its data type: max(int num1, int num2).
2. **Void Method Misuse:** Attempting to assign or print a void method call (e.g., int x = printGrade(85);) results in a compilation error.
3. **Missing Return Statement:** Value-returning methods MUST return a value along ALL conditional paths.
4. **Scope Leaks:** Local variables declared inside a method body cease to exist once execution leaves that method.

6. HIGH-YIELD EXAM TIPS & SHORTCUTS

- **Method Signature:** Defined strictly by Method Name + Parameter List (Type, Order, Number). Return type and modifiers are NOT part of the method signature.
- **Standard Library Methods:** Memory retention candidates: Math.pow(a, b), Math.random(), System.exit(0).
- **Pass-by-Value:** Java copies the actual argument value into the formal parameter variable upon invocation.

7. SELF-ASSESSMENT QUICK CHECK

1. **Q: What constitutes a method signature in Java?**
Ans: The method name and its parameter list (data types, number, and order).
2. **Q: Identify the error: public static void test(double a, b) {}**
Ans: Syntax error. b lacks a type declaration. Must be (double a, double b).
3. **Q: What is the output of Math.random() parameter requirement?**
Ans: It takes zero parameters and returns a double in range [0.0, 1.0).

1. METHOD OVERLOADING

Method Overloading allows a class to define multiple methods with the exact same name, provided that their **method signatures are different**.

Key Principles of Overloading:

- **Method Signature:** Consists of the method name and its parameter list (number of parameters, parameter types, and order of types).
- **Compiler Resolution:** The Java compiler determines which overloaded method to invoke at compile-time based on the arguments passed during invocation.
- **Flexibility:** Allows operations that perform similar logic on different data types (e.g., integers vs. floating-point numbers) to share a uniform method name.

2. RECURSION FUNDAMENTALS

A **recursive method** is a method that calls itself directly or indirectly to break down a problem into smaller, identical sub-problems.

Anatomy of a Recursive Function:

1. **Base Case (Stopping Condition):** The simple case where the function can compute the result directly without making further recursive calls. Prevents infinite call loops.
2. **Recursive Step:** The portion where the method invokes itself with modified parameters that move closer to the base case.

Key Mathematical Formulas in Recursion:

1. Factorial ($n!$):

Mathematical Definition:

$$n! = \begin{cases} 1 & \text{if } n = 0 \text{ (Base Case)} \\ n \times (n-1)! & \text{if } n > 0 \text{ (Recursive Step)} \end{cases}$$

2. Fibonacci Sequence (F_n):

Sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, ...

$$F(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$

3. COMPLETE CODE IMPLEMENTATION

Example 1: Method Overloading ('TestMethodOverloading.java')

```
public class TestMethodOverloading {
    public static void main(String[] args) {
        // Invokes max(int, int)
        System.out.println("Max of 3 and 4: " + max(3, 4));

        // Invokes max(double, double)
        System.out.println("Max of 3.0 and 5.4: " + max(3.0,
5.4));

        // Invokes max(double, double, double)
        System.out.println("Max of 3.0, 5.4, 10.14: "
+ max(3.0, 5.4, 10.14));
    }

    // Overload 1: Find max of two integers
    public static int max(int num1, int num2) {
        return (num1 > num2) ? num1 : num2;
    }

    // Overload 2: Find max of two doubles
    public static double max(double num1, double num2) {
        return (num1 > num2) ? num1 : num2;
    }

    // Overload 3: Find max of three doubles using nested call
    public static double max(double num1, double num2, double
num3) {
        return max(max(num1, num2), num3);
    }
}
```

Example 2: Recursive Factorial

```
public class TestFactorial {
    public static long factorial(int n) {
        if (n == 0) // Base Case
            return 1;
        else // Recursive Call
            return n * factorial(n - 1);
    }
}
```

Example 3: Recursive Fibonacci

```
public class TestFibonacci {
    public static long fib(int index) {
        if (index == 0) // Base Case 1
            return 0;
        else if (index == 1) // Base Case 2
            return 1;
        else // Reduction Step
            return fib(index - 1) + fib(index - 2);
    }

    public static void main(String[] args) {
        int n = 7;
        System.out.println("Fibonacci at index " + n
+ " is " + fib(n)); // Returns 13
    }
}
```

⚠ Common Student Pitfalls & Exam Traps:

- **Return Type Ambiguity:** Changing *only* the return type (e.g., `public static int max(int a, int b)` vs `public static double max(int a, int b)`) does **NOT** constitute method overloading! This results in a compiler error.
- **Infinite Recursion / StackOverflowError:** Missing the base case or executing a reduction step that fails to converge toward the base case leads to runtime memory exhaustion.
- **Ambiguous Calls:** Automatic type promotion (e.g., `int` to `double`) can cause compiler errors if two overloaded methods match equally well (e.g., `max(int, double)` vs `max(double, int)` when called with `(3, 4)`).

💡 High-Yield Exam Tips & Revision Shortcuts:

- **Nested Recursion Reuse:** Notice how `max(a, b, c)` reuses `max(max(a, b), c)`. In exams, look for existing methods to simplify higher-arity methods!
- **Fibonacci Call Tree:** Drawing a call tree diagram is the fastest way to trace recursive calls step-by-step during exam questions.
- **Signature Rule:** Signature = Method Name + Parameter Type List. Return types and modifier keywords (like `static`) are excluded.

4. SELF-ASSESSMENT QUICK CHECK

1. **Q: Is `void display(int a)` and `int display(int a)` valid overloading?**
Ans: No. The signature depends solely on method name and parameter types. Changing only the return type causes a duplicate method error.
2. **Q: What is the output of `factorial(4)`?**
Ans: $4 \times 3 \times 2 \times 1 \times 1 = 24$.
3. **Q: What value is returned by `fib(6)`?**
Ans: Index 6 corresponds to 8 in sequence (0, 1, 1, 2, 3, 5, 8).

09. Arrays & 1D Data Structures

1. CORE THEORY & FUNDAMENTALS

An **Array** is a data structure representing a fixed-size collection of elements of the *same data type* stored **contiguously in memory**.

- **Bulk Storage:** Replaces hundreds of discrete variables (e.g., name1, name2) with a single identifier (e.g., names).
- **Indexing:** Zero-based numerical index ($0 \leq \text{index} \leq N - 1$) for $O(1)$ random memory access.
- **Efficiency:** Process thousands of elements with 3 lines of loop code. Real-world applications: Playlists, leaderboards, inventory systems.

Array Variable Declaration

Declaring an array creates a reference variable in stack memory; it **does not allocate heap memory for elements** (initial value is null).

```
// Preferred Java Syntax:
elementType[] arrayRefVar;

// Alternative C-Style Syntax:
elementType arrayRefVar[];

// Example:
double[] myList; // Reference to double array
```

Creation & Heap Memory Allocation

Memory space is allocated explicitly using the new operator specifying size N ($N \geq 0$).

```
arrayRefVar = new elementType[arraySize];

// Combined Declaration and Creation:
double[] myList = new double[10];
```

Memory Layout & Default Initial Values

Upon creation via new, elements are auto-initialized to standard type defaults in contiguous heap memory blocks:

Data Type	Default Initial Value
byte, short, int, long	0
float, double	0.0
boolean	false
char	'\u0000' (Unicode null)
Object / String	null

Array Size & Accessing Elements

Array size is fixed at allocation and queried via arrayRefVar.length (read-only field).

```
int[] myList = new int[5]; // Indices: 0 to 4
myList[2] = 56;           // Assign 56 to 3rd element
myList[2] = myList[0] + 10; // Read and update value
```

Shorthand Array Initializers

Combines declaration, creation, and assignment without explicit new keywords:

```
double[] myList = {1.9, 2.9, 3.4, 3.5};
// Automatically evaluates length = 4
```

2. STANDARD ARRAY ALGORITHMS

1. User Input & Random Initialization

```
java.util.Scanner input = new java.util.Scanner(System.in);
for (int i = 0; i < myList.length; i++) {
    myList[i] = input.nextDouble(); // Reading input
}
for (int i = 0; i < myList.length; i++) {
    myList[i] = Math.random() * 100; // Random [0.0, 100.0)
}
```

2. Displaying Arrays & Special char[] Printing

```
for (int i = 0; i < myList.length; i++) {
    System.out.print(myList[i] + " ");
}
// SPECIAL CASE: char[] prints directly as a string!
char[] city = {'D', 'a', 'l', 'l', 'a', 's'};
System.out.println(city); // Directly outputs: Dallas
```

3. Summing & Finding Maximum Element

```
double total = 0;
for (int i = 0; i < myList.length; i++) {
    total += myList[i];
}

double max = myList[0]; // Assume 1st element is max
for (int i = 1; i < myList.length; i++) {
    if (myList[i] > max) max = myList[i];
}
```

4. Random Shuffling (Fisher-Yates)

```
for (int i = myList.length - 1; i > 0; i--) {
    int j = (int)(Math.random() * (i + 1)); // 0 <= j <= i
    double temp = myList[i];
    myList[i] = myList[j];
    myList[j] = temp;
}
```

5. Circular Left Shift

```
double temp = myList[0]; // Retain 1st element
for (int i = 1; i < myList.length; i++) {
    myList[i - 1] = myList[i]; // Shift left
}
myList[myList.length - 1] = temp; // Move 1st to last
```

6. Foreach (Enhanced For) Loops

Sequentially traverses elements without index variables.

```
for (double e: myList) {
    System.out.println(e); // Cannot modify array elements!
}
```

3. PRACTICAL LECTURE EXERCISES

Exercise A: Count Items Above Average

```
double sum = 0;
for (double val : numbers) sum += val;
double average = sum / numbers.length;
int count = 0;
for (double val : numbers) {
    if (val > average) count++;
}
System.out.println("Above average count: " + count);
```

Exercise B: Cafeteria Poll Summarizer

```
int[] responses = {1,2,5,4,3,5,2,1,3,1,4,3,3,3,2,3,3,2,2,5};
int[] frequency = new int[6]; // Index 1 to 5
for (int i = 0; i < responses.length; i++) {
    frequency[responses[i]]++; // Count frequency
}
for (int rating = 1; rating <= 5; rating++) {
    System.out.println(rating + "\t" + frequency[rating]);
}
```

4. ARRAYS IN METHODS (PASS-BY-SHARING)

Java passes arguments by **pass-by-value**. When passing an array, the **reference value (memory address)** is copied. This mechanism is termed **Pass-by-Sharing**: both caller and method reference the same physical heap array.

Primitive vs. Array Parameter Behavior

```
public class Test {
    public static void main(String[] args) {
        int x = 1;
        int[] y = new int[10];
        m(x, y);
        System.out.println("x is " + x + ", y[0] is " + y[0]);
        // Output: x is 1 (Unchanged), y[0] is 5555 (Mutated!)
    }
    public static void m(int number, int[] numbers) {
        number = 1001; // Modifies local stack copy only
        numbers[0] = 5555; // Mutates original array in heap!
    }
}
```

Swapping Elements Trap & Fix

```
// FAILS: Swaps local primitive variable copies!
public static void swap(int n1, int n2) {
    int temp = n1; n1 = n2; n2 = temp;
}

// SUCCEEDS: Modifies actual array indices in heap!
public static void swapFirstTwoInArray(int[] array) {
    int temp = array[0];
    array[0] = array[1];
    array[1] = temp;
}
```

Anonymous Arrays & Returning Array References

```
// Anonymous Array (no reference variable):
printArray(new int[]{3, 1, 2, 6, 4, 2});

// Returning a newly allocated reversed array:
public static int[] reverse(int[] list) {
    int[] result = new int[list.length];
    for (int i = 0, j = result.length - 1; i < list.length; i++, j--) {
        result[j] = list[i];
    }
    return result;
}
```

5. MULTI-DIMENSIONAL ARRAYS

A 2D array is an **array of 1D arrays**. An N-dimensional array generalizes to $N \geq 3$.

2D Array Creation & Ragged Arrays

```
int[][] matrix = new int[5][5];
matrix[2][1] = 7; // Row index 2, Column index 1

// Shorthand Initialization:
int[][] array = {
    {1, 2, 3}, // Row 0
    {4, 5, 6}, // Row 1
    {7, 8, 9} // Row 2
};

// Ragged Array (varying row lengths):
int[][] x = new int[3][];
x[0] = new int[2]; x[1] = new int[4];
```

Processing 2D Arrays with Nested Loops

```
// Summing all elements in a 2D Matrix:
public static int sum(int[][] m) {
    int total = 0;
    for (int row = 0; row < m.length; row++) {
        for (int col = 0; col < m[row].length; col++) {
            total += m[row][col];
        }
    }
    return total;
}
```

3D Arrays Structure

Represents 3D structures (e.g., 4 students, 5 exams, 2 parts [MCQ, Essay]):

```
double[][][] scores = new double[4][5][2];
scores[0][2][1] = 33.5; // Student 0, Exam 2, Part 1
```

6. EXAM CASE STUDY: TEST GRADING

```
char[][] answers = {
    {'A','B','A','C','C','D','E','E','A','D'}, // Student 0
    {'D','B','A','B','C','A','E','E','A','D'} // Student 1...
};
char[] key = {'D','B','D','C','C','D','A','E','A','D'};

for (int i = 0; i < answers.length; i++) {
    int correctCount = 0;
    for (int j = 0; j < answers[i].length; j++) {
        if (answers[i][j] == key[j]) correctCount++;
    }
    System.out.println("Student " + i + "'s score: " + correctCount);
}
```

⚠️ COMMON STUDENT PITFALLS & EXAM TRAPS

- **Off-by-One Error:** Accessing `myList[myList.length]` triggers `ArrayIndexOutOfBoundsException`. Max valid index is `myList.length - 1`.
- **Fixed Array Size:** Array sizes cannot change after instantiation. Writing `myList.length = 10;` yields a compilation error.
- **Printing 1D Arrays directly:** Calling `System.out.println(myList)` prints the hash code pointer (e.g., `[I@15db9742]`, EXCEPT for `char[]`!)

💡 HIGH-YIELD EXAM TIPS & REVISION SHORTCUTS

- `array.length` is a **field** (NO parentheses). Compare with `String.length()` which is a **method**!
- **2D Dimensions:** `m.length` = number of rows; `m[i].length` = number of columns in row `i`.
- **Foreach limitation:** `for (double e: list)` cannot modify elements inside the original array!

📄 SELF-ASSESSMENT QUICK CHECK

1. What is default value of `boolean[] b = new boolean[3];`?
Ans: false.
2. What is printed by `int[] a = {1, 2}; swap(a[0], a[1]);`?
Ans: `a[0]=1, a[1]=2` (Primitives passed by value).
3. How do you find column count of row 2 in 2D array `m`?
Ans: `m[2].length`.
4. Valid anonymous array creation syntax:
Ans: `new int[]{5, 10, 15};`